

Subrutinas y Abstracción de Control

Abstracción

Proceso a través del cual el programador puede asociar un nombre con un fragmento del programa. La idea es pensar la misma en términos de su propósito o funcionalidad en lugar de en términos de su implementación.

- **De control**

Su propósito es definir una operación bien definida

- **De datos**

Su propósito es representar información
(notar que en general para contar con abstracción de datos hace falta contar con abstracción de control)

Control de secuencia

Las estructuras de control de secuencia pueden categorizarse en cuatro grupos:

- **Expresiones:** Las expresiones son la forma básica de construir bloques y expresan como los datos son manipulados y modificados por el programa.
- **Sentencias o instrucciones:** determinan como el control fluye de un segmento de programa a otro.
- **Programación declarativa:** modelo de ejecución que no depende de sentencias, pero establece como la ejecución tiene lugar (ejemplo Prolog).
- **Unidades-Subprogramas:** indican una manera de pasar transferir el control de un segmento de programa a otro.

Esta división no es precisa ya que hay lenguajes que tiene más de una característica, por ejemplo LISP (computa con expresiones pero usa mecanismos de control de sentencias)

Control de secuencia

Las estructuras de control de secuencia pueden ser:

- **Implícitas:** son las definidas por el lenguaje para tener efecto a menos que sean modificadas por el programador a través de una estructura de control explícita.
- **Explícitas:** son aquellas definidas por el programador para modificar el control implícito. Por ejemplo el uso de los paréntesis en las expresiones o el uso de la instrucción goto a un etiqueta en un programa que solo tiene secuencia.

En clases previas ya vimos de qué manera se resuelve el control de secuencia a través de expresiones e instrucciones. Veamos las dos alternativas que nos faltan.

Control de secuencia: Programación declarativa

Pattern Matching: una operación tiene éxito si ajusta y asigna un conjunto de variables a un patrón predefinido.

Es una operación crucial para lenguajes como: Perl, Snobol, ML, Miranda, Prolog.

El pattern matching produce dos efectos:

- Por un lado, como estructura de control, permite decidir qué parte del código se va a ejecutar.
- Permite establecer ligaduras entre el patrón de la definición y el de la instancia (instancia las variables).

En el caso de Prolog el mecanismo utilizado es más potente que el simple pattern matching, utiliza **unificación**.

Abstracción

Proceso a través del cual el programador puede asociar un nombre con un fragmento del programa. La idea es pensar la misma en términos de su propósito o funcionalidad en lugar de en términos de su implementación.

- **De control**

Su propósito es definir una operación bien definida



Subrutinas

- **De datos**

Su propósito es representar información
(notar que en general para contar con
abstracción de datos hace falta contar con
abstracción de control)

Control de secuencia: Unidades o subrutinas

Los lenguajes de programación imperativos y orientados a objetos brindan mecanismos para dividir el programa en **unidades**, cada una de las cuales tiene cierta coherencia y lógica.

Su incorporación fomenta:

- **Eficiencia**: permiten factorizar el código (en los primeros lenguajes solo se perseguía el incremento de eficiencia principalmente de almacenamiento).
- **Legibilidad**: metodología de diseño top-down (mecanismos más abstractos).
- **Verificación**: una unidad puede pensarse como un mapeo entre dominios de valores. Se intenta que los chequeos sean lo más seguros posibles, así evolucionaron los métodos de pasaje de parámetros.

Unidades o subrutinas

Las **unidades** pueden ser:

- Anónimas (bloques)
- Con nombre asociado (subprogramas)

Un unidad con nombre permite extender el lenguaje con una nueva primitiva.

Para que la abstracción sea poderosa, se debería poder usar conociendo sólo la interfaz. La implementación debería ser transparente.

Recordar que la unidad puede ser una **abstracción a nivel expresión** o una **abstracción a nivel instrucción**.

Unidades o subrutinas

En los lenguajes orientados a objetos cada unidad con nombre brinda un servicio que puede ser provisto por cualquiera de las instancias de la clase en la cuál se define la unidad. En este tipo de lenguajes la clase es el mecanismo de abstracción.

Un servicio puede acceder y hasta modificar el estado interno del objeto que recibe el mensaje que provoca la ejecución de ese servicio.

Aspectos de diseño a tener en cuenta por el LP

- Entidades
- Ligaduras
- Reglas de alcance
- Sistemas de tipos
- Soporte para definir unidades

Unidades o subrutinas

Aspectos de diseño específicos

- Estructura estática
- Recursividad
- Control
- Métodos de pasaje de parámetros
- Correspondencia entre parámetros formales y actuales
- Sobrecarga y polimorfismo
- Tipos de los parámetros: datos y unidades
- Ambiente de referenciamiento de las unidades que recibe como parámetro (chequeo estático o dinámico de unidades)

Unidades o subrutinas

Atributos de la unidad

- **Nombre:** algunos lenguajes permiten definir unidades anónimas. Las unidades anónimas se ejecutan implícitamente y cuando se alcanzan se crea un nuevo ambiente de referenciamiento.
- **Lista de parámetros:** permiten la comunicación de la unidad con el resto del programa. Si se admiten parámetros de diferentes tipos la unidad es polimórfica.
- **Ambiente de referenciamiento:** se establece a que otras unidades puede referenciar.
- **Bloque ejecutable:** si la unidad tiene la capacidad de llamarse a si misma soporta recursividad.
- **Alcance:** segmento de código dentro del cual la unidad puede invocarse (si incluye a la misma unidad acepta recursividad).

Unidades

Aspectos de implementación

- Chequeo de tipos entre parámetros
- Unidades de compilación
- Creación del ambiente de referenciamiento local

Estructura estática:

La unidad tiene un **encabezamiento** y un **cuerpo**.

- El encabezamiento está formado por el nombre de la unidad, la lista de parámetros y el tipo del resultado.
- El cuerpo está formado por las declaraciones locales y la sección ejecutable.

Unidades

Estructura estática:

En el encabezamiento se puede definir el **perfil de los parámetros**, el **protocolo** y la **signatura**.

- El perfil de los parámetros de una unidad es el número, orden y tipo de los parámetros formales.
- El protocolo de una unidad es el perfil de los parámetros más el tipo retornado por la unidad, si es que tiene.
- La signatura de una unidad está dada por el nombre de la unidad y su protocolo y define la interfaz de la unidad.

Unidades

La unidad estáticamente posee:

- Declaración
- Definición
- Invocación

La unidad dinámicamente puede estar:

- Pasiva
- Activa – Suspendida

Su estado está determinado por las estructuras de control.

La invocación tiene una vinculación bidireccional con la activación de la unidad y con su suspensión

Unidades

La **comunicación** entre unidades puede darse a través de:

- Ambiente global
- Ambiente implícito
- Parámetros y resultado

La comunicación a través del ambiente global puede generar efectos colaterales.

La comunicación a través del ambiente implícito puede generar comportamiento sensible a la historia.

Recordemos que hay efecto colateral si la ejecución de una unidad modifica el ambiente de referenciamiento de la unidad llamadora

Mecanismos para definir unidades

- **Bloques:** unidades anónimas

- **Rutinas:**

- **Abstracciones procedurales: abstracción a nivel instrucción**

La manera de invocar a este tipo de rutinas es a través de una llamada en la cual aparece el nombre de la rutina:

```
call <nombre proced>
```

- **Abstracciones funcionales: abstracción a nivel expresión**

La manera de invocar a este tipo de rutinas es en el contexto de una expresión.

Mecanismos para definir unidades

Una **abstracción procedural** es una unidad que al ser invocada evalúa una instrucción y en general provoca un cambio en el estado de computación, esto es, modifica los valores del ambiente de referenciamiento de la llamada provocando un efecto colateral.

El usuario de un procedimiento observa únicamente las transformaciones en estado de la memoria pero no los pasos que dieron lugar a esta transformación.

Una **abstracción funcional** es una unidad que evalúa una expresión y al ser invocada produce un resultado. Idealmente no debería producir un efecto colateral. Es una manera de hacer abstracción a nivel expresión.

Mecanismos para definir unidades: funciones

En lenguajes imperativos

```
Function Pot(n,m:int):int
    if n=1 then Pot:=1
        else Pot:= Pot(n,m-1)*n
End;
```

En lenguajes funcionales

```
fun Pot(n,m:int):int
    if n=1 then 1
        else Pot(n,m-1)*n
End;
```

En algunos lenguajes se hace el **return** del valor (El valor computado no se asigna a una variable)
Se utiliza el nombre la función como área de almacenamiento.

Así el nombre de la función tiene dos semánticas dependiendo de dónde aparezca:

- Área de almacenamiento auxiliar (a la izquierda)
- Invocación recursiva (a la derecha)

En los lenguajes funcionales se evita la doble semántica sobre el nombre de la función.

Representación en memoria

El espacio que se aloca para una subrutina en la pila se conoce como **registro de activación** o *stack frame*.

El registro contiene los argumentos, valores de retorno (en caso que los tenga), variables locales y temporales.

El registro se elimina de la pila cuando la subrutina alcanza su *epílogo*.

Hay dos punteros que mantienen el estado de la pila: **actual** (*frame pointer*) y **libre** (*stack pointer*)

Los objetos del registro de activación se acceden por desplazamiento desde la posición almacenada en **actual**.

Representación en memoria

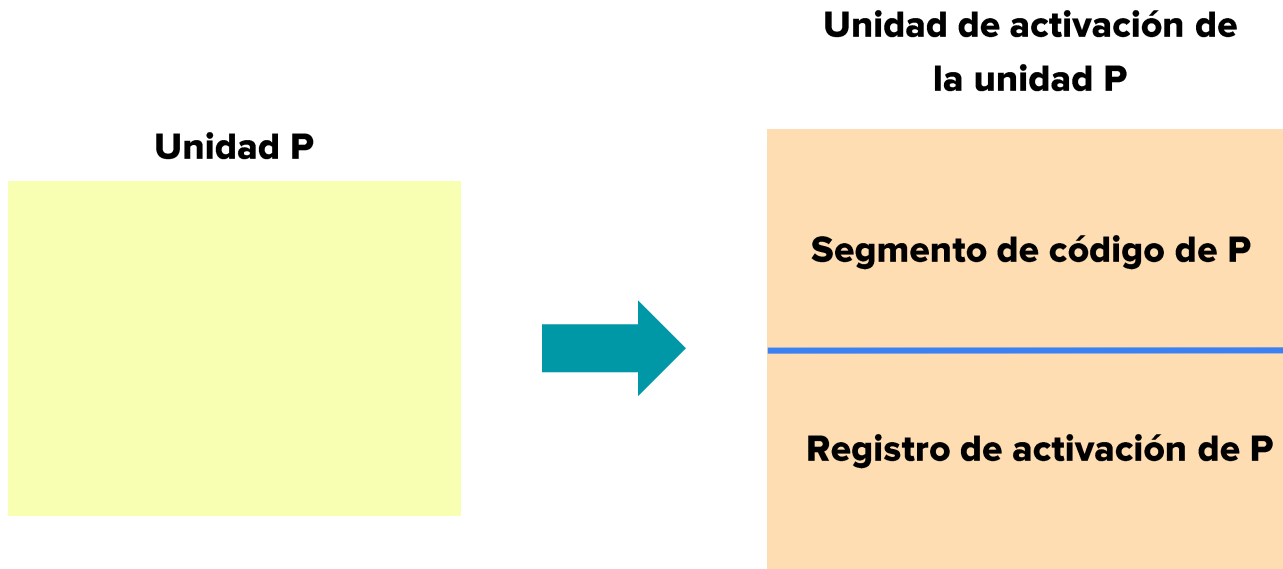
El espacio que se aloca para una subrutina en la pila se conoce como **registro de activación** o *stack frame*.

El registro de activación mantiene además información de sistema para garantizar el funcionamiento: puntero de retorno, enlace dinámico...

En aquellos lenguajes que admiten anidamiento, es necesario conservar la cadena estática a fin de resolver los accesos no locales (enlace estático)

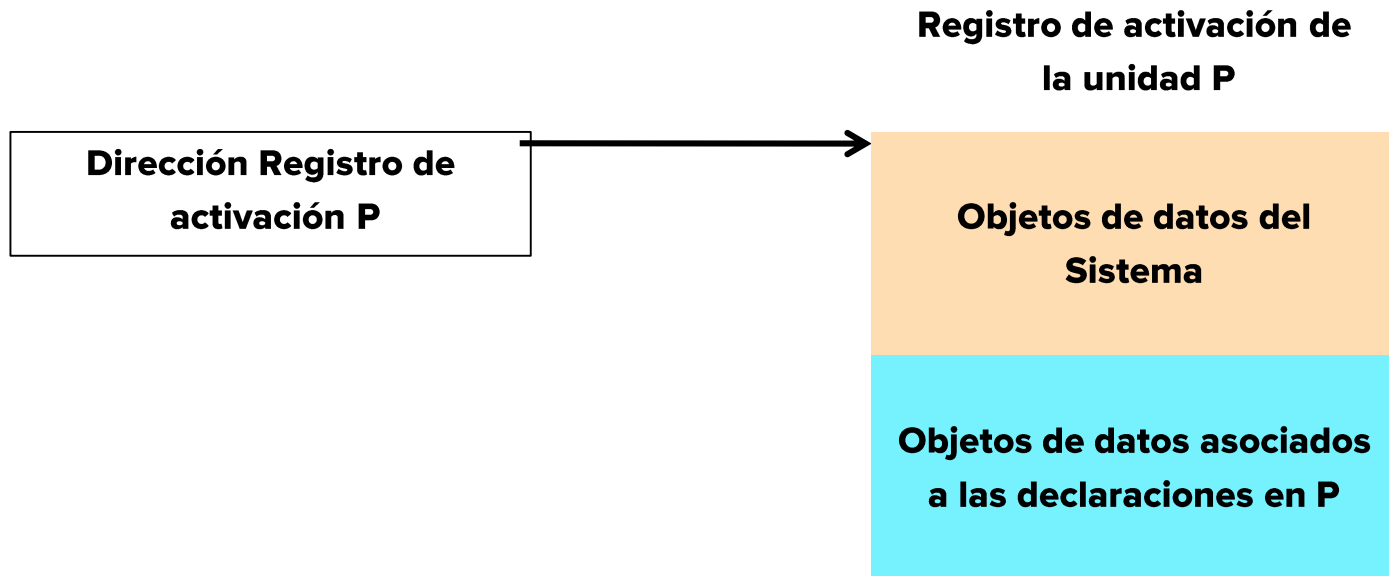
Representación en memoria

Si nuestro lenguaje considera unidades, nuestro procesador abstracto debe considerar la *unidad de activación de P*. La unidad de activación contiene toda la información relevante en ejecución de la unidad.



Representación en memoria

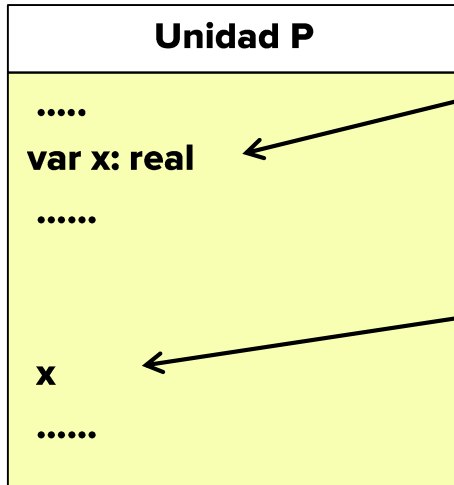
El *registro de activación de P* contiene la información sobre los objetos de datos, del sistema y declarados en la unidad.



Representación en memoria

El acceso a cualquier objeto de dato declarado en P, se calcula como:

$\text{Dirección Registro de Activación P} + \text{desplazamiento de la variable}$



En la declaración se determina el desplazamiento de la variable x.

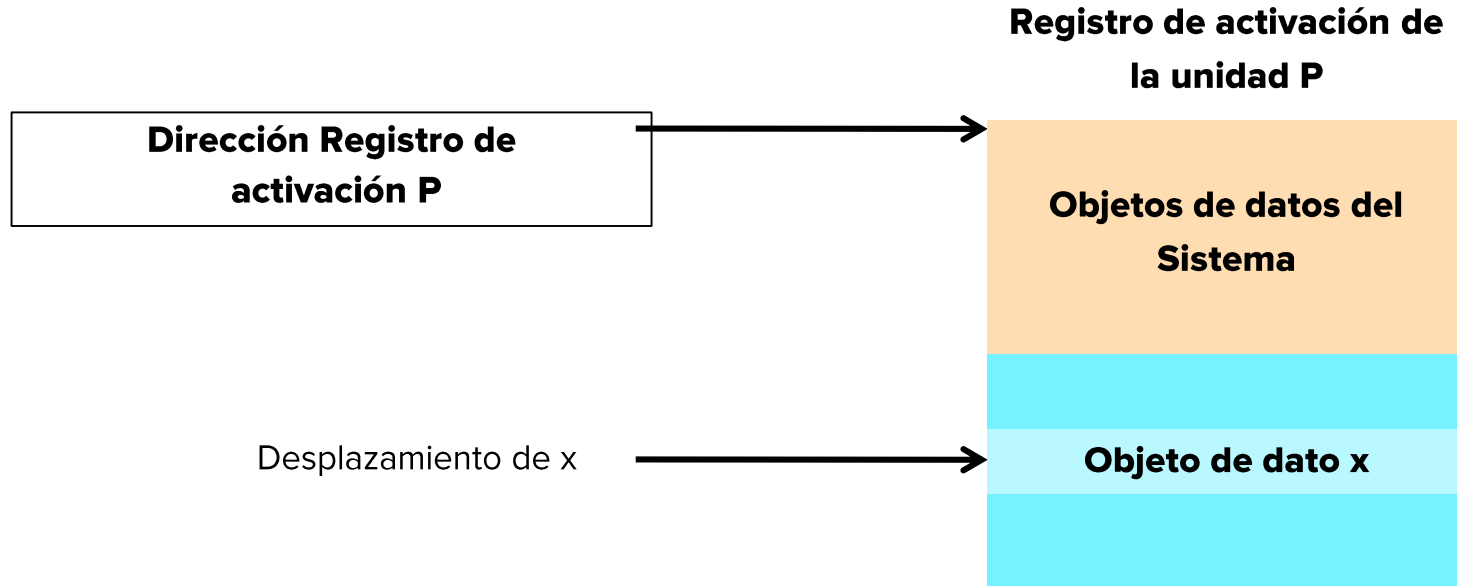
En el uso se accede a la misma mediante la fórmula:

$\text{Dir. RA P} + \text{desplazamiento de x}$

Representación en memoria

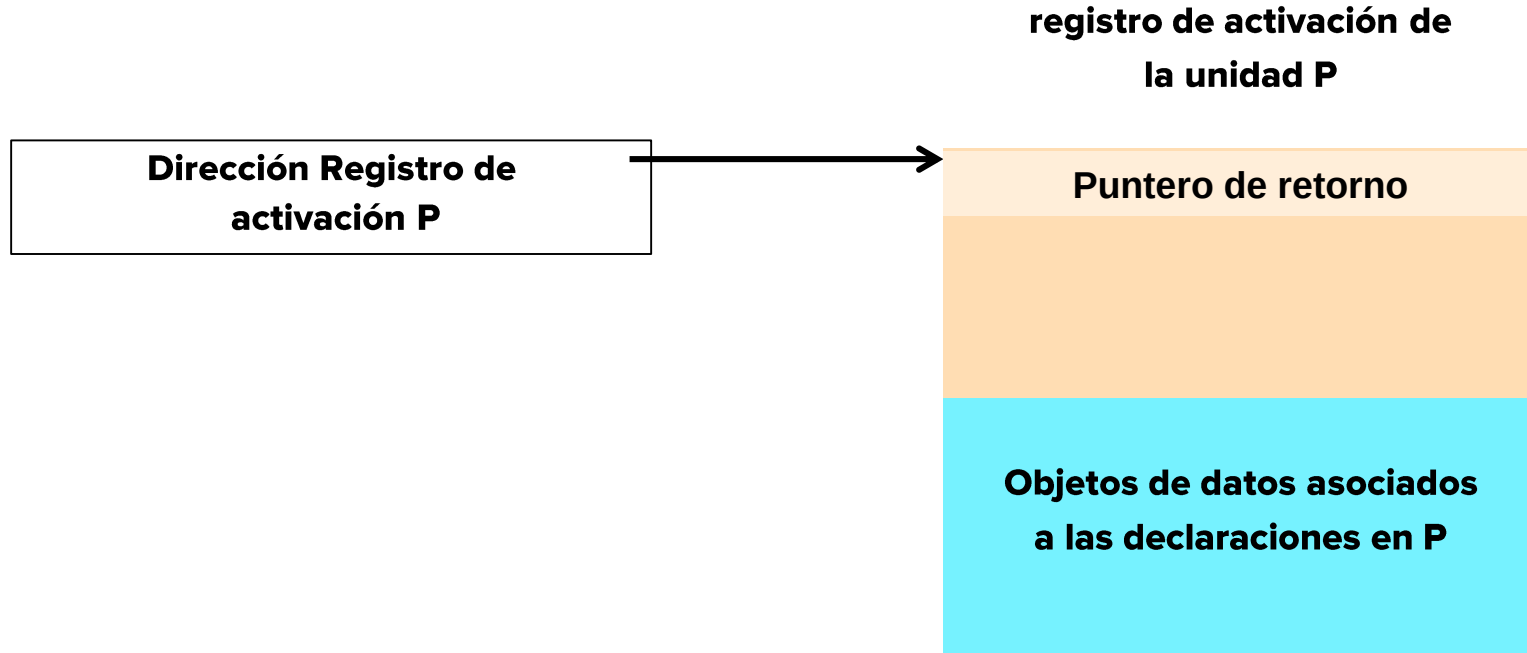
El acceso a cualquier objeto de dato declarado en P, se calcula como:

Dirección Registro de Activación P + desplazamiento de la variable



Representación en memoria

Si tratamos con unidades simples, sin parámetros, el único objeto del sistema que resulta imprescindible guardar es el puntero de retorno.



Representación en memoria

Los lenguajes de programación puede ser clasificados de acuerdo las decisiones que toman en diseño. Se puede clasificar de acuerdo al manejo de memoria y a las regla de alcance.

La clasificación de acuerdo al manejo de memoria que utilizan nos conduce a lenguajes:

- Estático
- Basado en pila
- Dinámico

Representación en memoria

Por otro lado la clasificación de acuerdo a las reglas de alcance y al esquema de manejo de memoria:

- Alcance estático
 - Alocación estática
 - Alocación semiestática (basada en pila)
 - Alocación semidinámica (basada en pila)
 - Alocación dinámica (basada en pila)
- Alcance dinámico

Finalmente si tomamos en cuenta la resolución de invocaciones a objetos de datos

- Acceso local
- Acceso global
- Esquema de alocación

Parámetros

Hay dos maneras de hacerle llegar los datos sobre los cuales computar a una unidad:

- A través del acceso a variables no locales visibles para la unidad
- A través del pasaje de parámetros

Los datos pasados como parámetros son accedidos a través de nombres que son locales a la unidad.

El pasaje de parámetros es más flexible, ya que define computación parametrizada.

El uso de variables no locales puede conducir a programas menos confiables

Parámetros: terminología

- **Argumento:** datos que forman la entrada de una unidad: datos no locales, parámetros y archivos externos (idealmente todos deberían ser parámetros)
- **Resultado:** variables no locales o archivos externos que también pueden modificarse además de los parámetros (se mantienen en el área de almacenamiento temporal , para las funciones no se retornan a través de los parámetros)
- **Parámetro formal:** dato particular dentro del área de referenciamiento de la unidad. Se declaran en el encabezado de la unidad; aclarando en general su tipo
- **Parámetro actual:** variables, constantes, expresiones. Pueden ser locales, no locales o a su vez parámetros.

La elección de qué se permite como parámetro actual afecta la implementación del lenguaje de programación.

Tipos de parámetros

El lenguaje puede ser más o menos ortogonal dependiendo de qué entidades pueden ser parámetro.

- ¿Todas las entidades pueden ser parámetros?
- ¿Cualquier dato puede ser argumento y/o resultado?

Pasaje de parámetros

Hay diferentes semánticas que los lenguajes pueden elegir para el pasaje de parámetros:

Algunas decisiones importantes:

- Forma de evaluación de los parámetros actuales
- Modo del pasaje: in, out o in-out

Modo in:

el parámetro formal recibe un valor y lo usa como una constante dentro del cuerpo de la unidad que lo define

Modo out:

el parámetro formal funciona como una como una variable local, cuyo valor final se “copia” al parámetro actual

Modo in-out:

el parámetro formal funciona como un vehículo para recibir un valor, pero también como una manera de extraer un valor como resultado de la ejecución.

Pasaje de parámetros

Clasifiquémolas según la evaluación a la que están asociados:

Evaluación Ansiosa

Los parámetros actuales se evalúan al momento de la llamada

- Pasaje de parámetros por referencia (in/out)
- Pasaje de parámetros por copia:
 - Valor (in)
 - Resultado (out)
 - Valor-Resultado (in/out)

Evaluación de orden normal

Los parámetros actuales se evalúan al momento de su uso

- Pasaje de parámetros por nombre

Evaluación Perezosa

Los parámetros actuales se evalúan al momento de su uso, solo la primera vez

- Pasaje de parámetros por necesidad

Unidades: Estructura de control

Formas de estructurar el control sobre unidades:

- Jerárquicas
- Simétricas
- Excepciones y eventos
- Planificadas
- Paralelas

Corrutinas

- **Creación:** deben realizarse las operaciones de inicialización
- **Detach:** luego de independiza (detach) del programa principal. Esta operación crea un objeto al cual se le puede transferir la ejecución luego y retorna una referencia a su llamador.
- **Transferencia:** guarda el valor concreto del program counter en el objeto creado en el paso previo y reinicia la corrutina indicada como parámetro. El programa principal juega el papel de corrutina default inicial

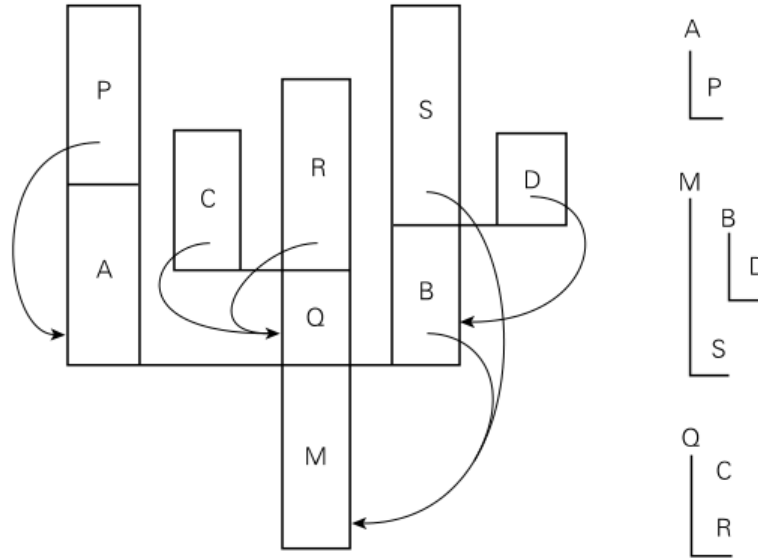
```
Coroutine check_file_sys
Inicializar
Detach
For all files
    ...
    transfer(us)
```

```
Coroutine Update_screen
Inicializar
Detach
Loop
    ...
    tranfer(cfs)
```

```
Begin // main
    us:= new update_screen
    cfs:= new check_file_sys
    transfer(us)
```

Corrutinas

El manejo de la memoria se vuelve más complejo al utilizar corrutinas. Veamos un ejemplo



Concurrencia

Debe implementarse el constructor `and`

- **Opción 1:** Si la ejecución puede ser paralela, no se hace ningún tipo de suposición sobre el orden de ejecución. Por lo que tranquilamente se podría ejecutar en secuencia
- **Opción 2:** La otra alternativa es utilizar primitivas de ejecución paralela a nivel de sistema operativo (`fork`)

Ejemplo: Call A **and** Call B **and** Call C;

Opción 1:

```
While moreToDo do
  moreToDo := false;
  call A;
  call B;
  call C;
end
```

Opción 2:

```
fork A;
fork B;
fork C;
wait
```

Iteradores

Teniendo en cuenta la implementación de corrutinas, la implementación de iteradores es casi trivial: una corrutina representa el programa principal; una segunda corrutina se utiliza para representar al iterador.

- **start()**: que inicializa el bloque posicionando el cursor en el primer elemento de la estructura
- **more()**: que determina si hay un próximo elemento en la estructura.
- **next()**: que se posiciona en el próximo elemento de la estructura si es que hay alguno

Iteradores: ejemplo

```
class ParesIterador:
    def __init__(self, limite):
        self.limite = limite
        self.valor = 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.valor > self.limite:
            raise StopIteration
        else:
            resultado = self.valor
            self.valor += 2
            return resultado
```

```
# Crear una instancia del iterador
iterador = ParesIterador(10)

# Usar el iterador en un bucle for
for par in iterador:
    print(par)
```

Resultado ejecución:

```
0
2
4
6
8
10
```

¿Preguntas?
